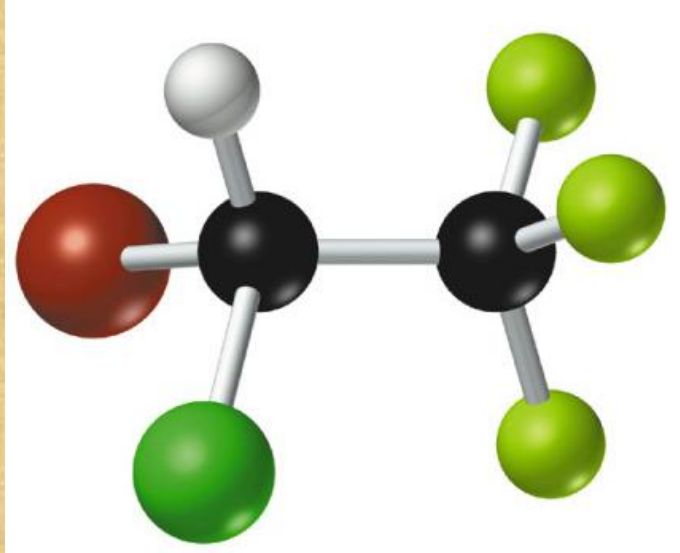
Expert Systems

Presented by Ulit Jaidee

What is an expert system?

- An **expert system** uses knowledge <u>specific</u> to a problem domain to provide “expert quality” to perform in a <u>specific</u> application area
- An **expert system** is an interactive computer-based decision tool that uses both facts and rules to solve difficult decision problems based on knowledge acquired from an expert.

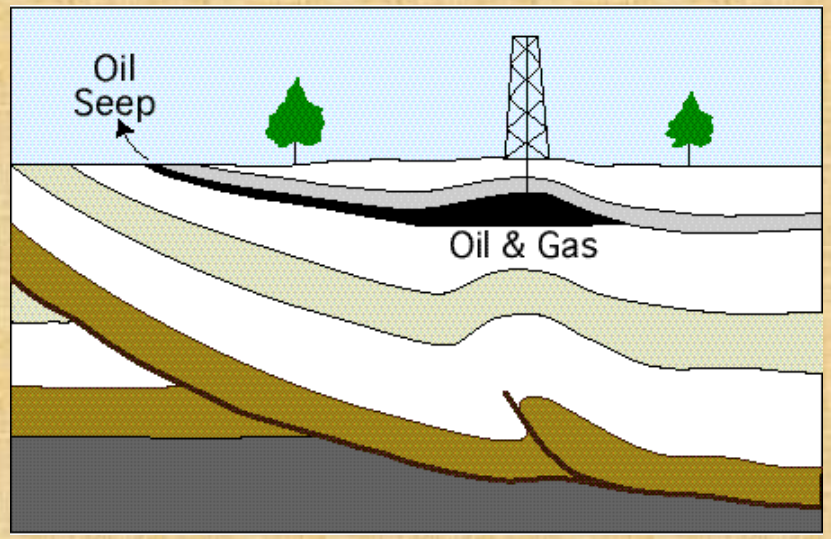
Applications of Expert Systems



DENDRAL: Used to identify the structure of chemical compounds. First used in 1965

MYCIN
Medical system for diagnosing blood disorders. First used in 1979

Applications of Expert Systems



PROSPECTOR:
Used by geologists to identify sites for drilling or mining. First used in 1978
(found a $100M ore deposit)

XCON
Configure VAX and PDP-11 series.
First used in 1984
(saved DEC $70M per year)

Applications of Expert Systems

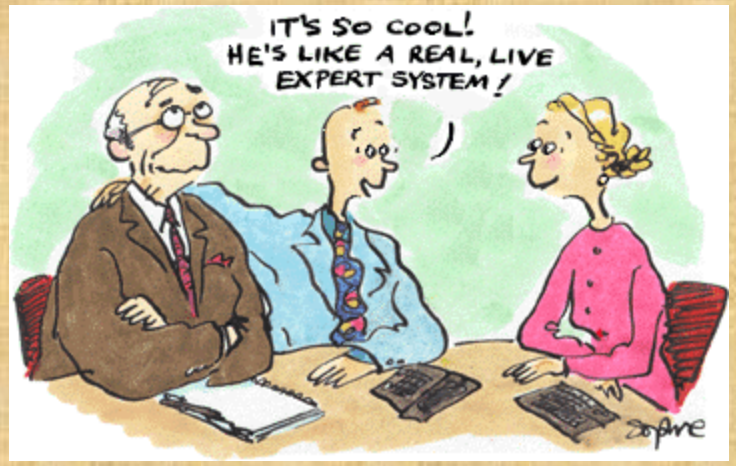
LITHIAN: Gives advice to archaeologists examining stone tools

PUFF:
Medical system
for diagnosis of respiratory conditions. Fist used in 1979

Categories of Expert Systems

Most applications of expert systems will fall into one of the following categories:

- Interpreting and identifying
- Predicting
- Diagnosing
- Designing
- Planning
- Monitoring
- Debugging and testing
- Instructing and training
- Controlling



Why use Expert Systems?

- Human experts are not always available. An expert system can be used anywhere, any time.
- Human experts are not 100% reliable or consistent
- Human experts may not be good at explaining decisions
- Cost effective

Problems with Expert Systems

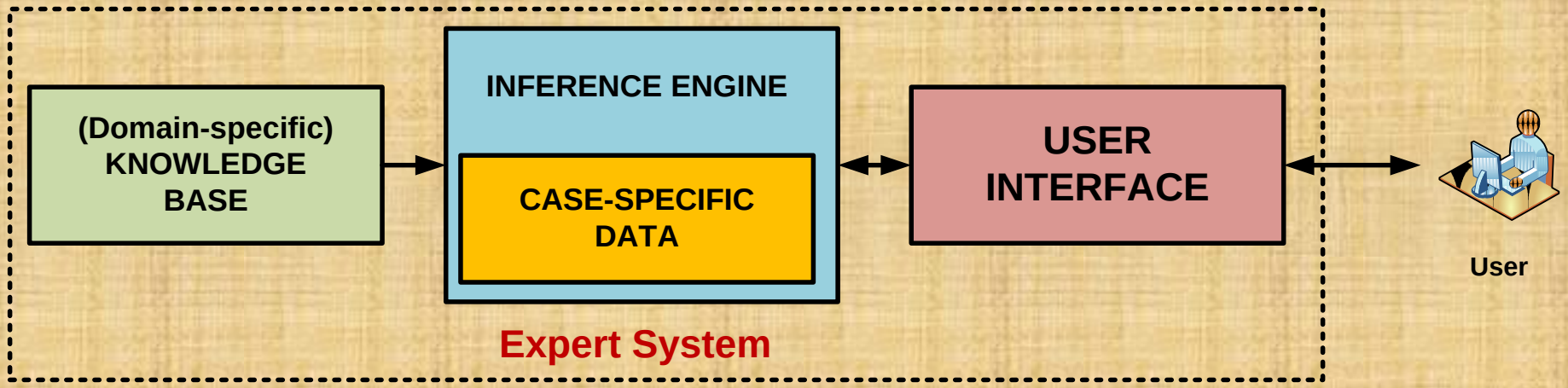
- Limited domain
- Systems are not always up to date, and do not learn
- No “common sense”
- Experts (or engineers) needed to setup and maintain system

Common characteristics

- System performs at a level generally recognized as equivalent to a human expert in the field
- System is highly domain specific
- System can explain its reasoning

System Architecture

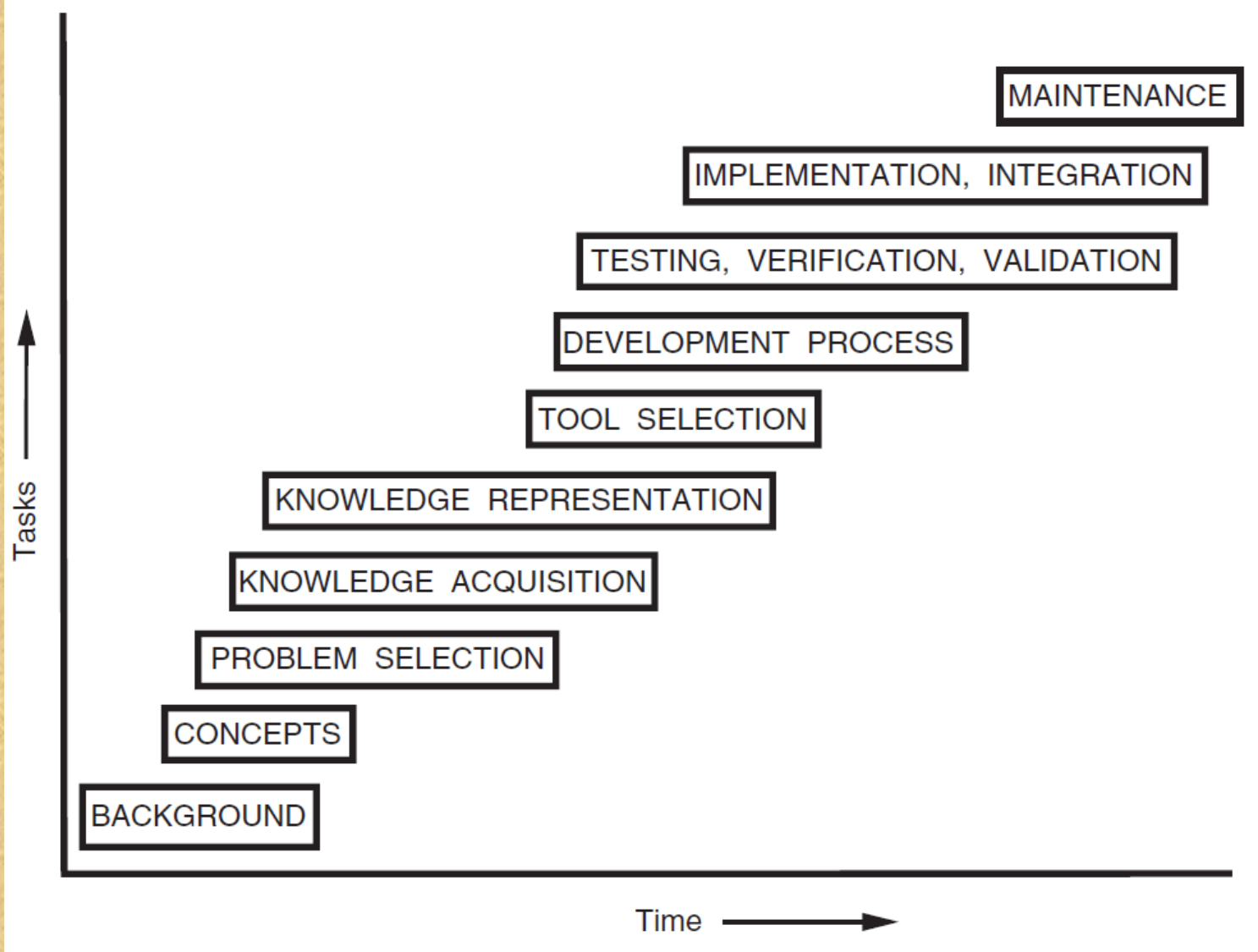
- Usually expert systems are rule-based
  - Extract expert knowledge in the form of rules & facts



System Architecture (cont')

- The **knowledge base** is the collection of facts and rules which describe all the knowledge about the problem domain
- The **inference engine** is the part of the system that chooses which facts and rules to apply when trying to solve the user's query
- The **user interface** is the part of the system which takes in the user's query in a readable form and passes it to the inference engine. It then displays the results to the user.
- The **case-specific data** is working memory

Development Process



Knowledge Representation

- Traditionally the most popular form of knowledge representation in expert systems is **production rules**.

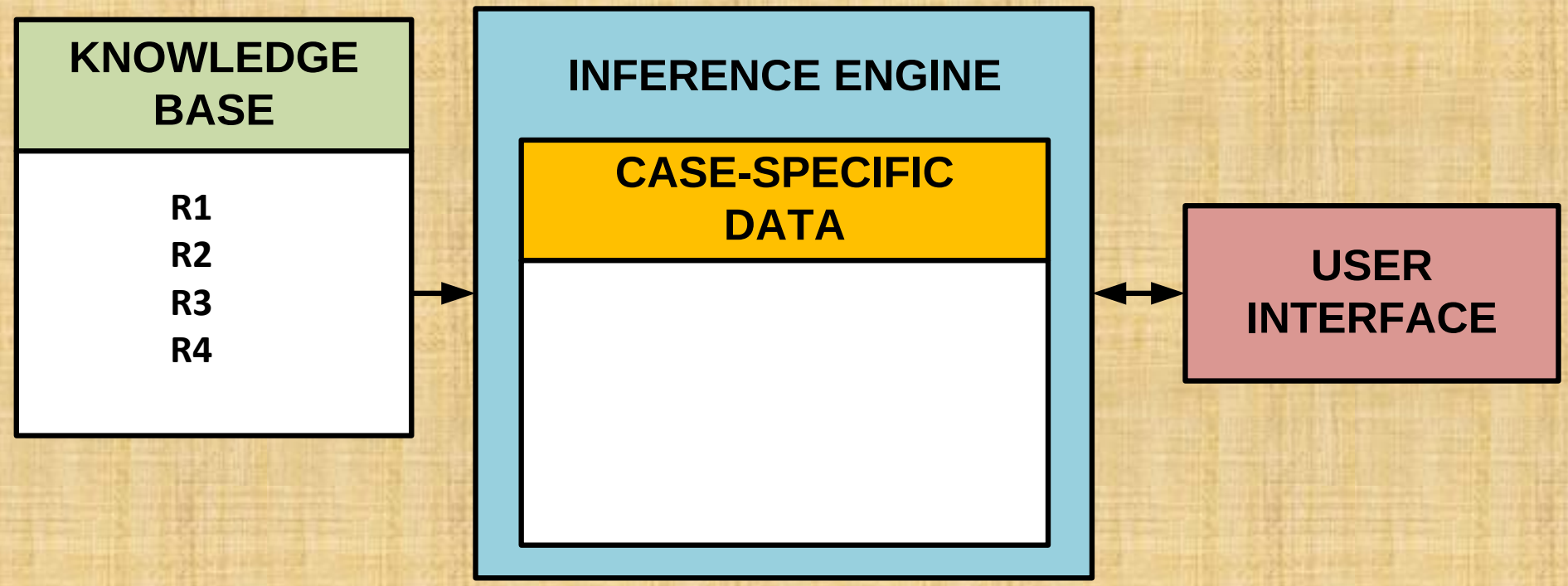
- Examples of rules:
  - if condition P then conclusion C
  - if situation S then action A
  - if conditions C_1 and C_2 hold then conclusion C does not hold

Inference example

- Consider the following rules about diagnosing auto problems

Rule#	Condition	Consequence
R1	If gas_in_engine and turns_over	Then problem(spark_plugs)
R2	If not (turns_over) and not(lights_on)	Then problem(battery)
R3	If not(turns_over) and lights_on	Then problem(starter)
R4	If gas_in_tank and gas_in_carb	Then gas_in_engine

Inference example (cont')



Example rule from MYCIN

if

1 the infection is primary bacteremia, and

2 the site of the culture is one of the sterilesites, and

3 the suspected portal of entry of the organism is the gastrointestinal tract

then

there is suggestive evidence (0.7) that the identity of the organism is bacteroides.

Example rules from AL/X

- Diagnosing equipment failure on oil platforms, Reiter 1980

if

the pressure in V-01 reached relief valve lift pressure

then

the relief valve on V-01 has lifted [N = 0.005, S = 400]

if

NOT the pressure in V-01 reached relief valve lift pressure,

and the relief valve on V-01 has lifted

then

the V-01 relief valve opened early (the set pressure has drifted)

[N = 0.001, S = 2000]

Example rule from AL3

game playing, bratko 1982

if

1 there is a hypothesis, *H*, that a plan *P* succeeds, and

2 there are two hypotheses,

***H*1, that a plan *R*1 refutes plan *P*, and**

***H*2, that a plan *R*2 refutes plan *P*, and**

3 there are facts: *H*1 *is false*, and *H*2 *is false*

then

1 generate the hypothesis, *H*3, that the combined

plan '*R*1 *or R*2' refutes plan *P*, and

2 generate the fact: *H*3 *implies not*(*H*)

Inference Methods

Forward vs. **Backward chaining**

- Forward chaining or **Data driven reasoning** – an inference technique which uses IF THEN rules to deduce a problem solution from initial data
- Backward chaining or **Goal driven reasoning** – an inference technique which uses IF THEN rules to repetitively break a goal into smaller sub-goals which are easier to prove

Forward chaining

- Idea: fire any rule whose premises are satisfied in the *KB*,
  - add its conclusion to the *KB*, until query is found

$$R_1 : P \Rightarrow Q$$

$$R_2 : L \wedge M \Rightarrow P$$

$$R_3 : B \wedge L \Rightarrow M$$

$$R_4 : A \wedge P \Rightarrow L$$

$$R_5 : A \wedge B \Rightarrow L$$

$$F_1 : A$$

$$F_2 : B$$

Backward chaining

Idea: work backwards from the query *q*

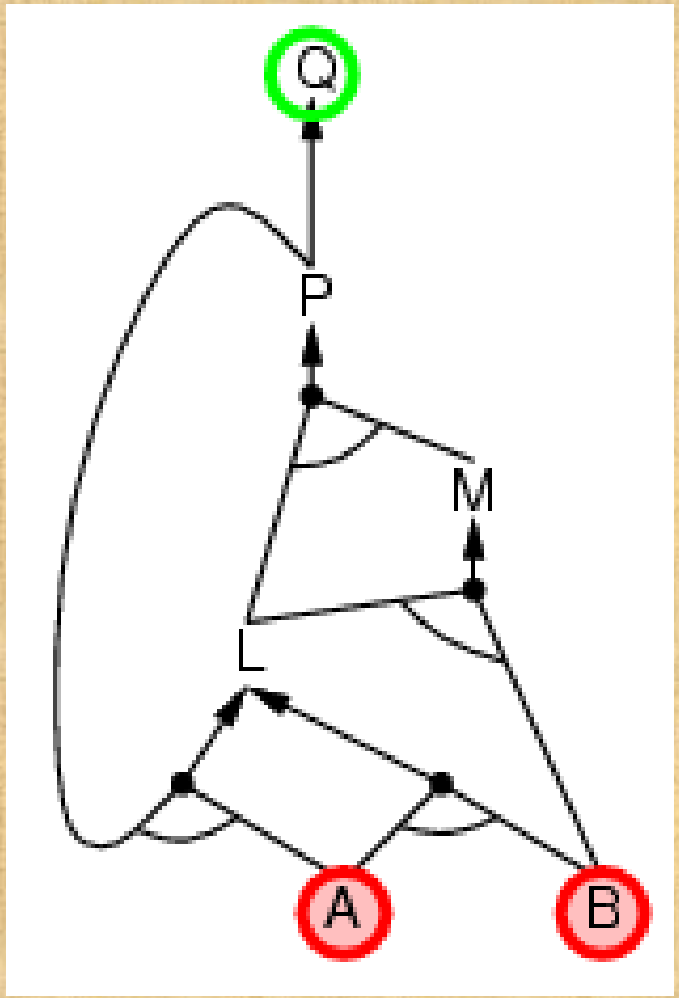
- check if *q* is known already, or
- prove by BC all premises of some rule concluding *q*
- Hence BC maintains a stack of sub-goals that need to be proved to get to q.

Avoid loops: check if new sub-goal is already on the goal stack

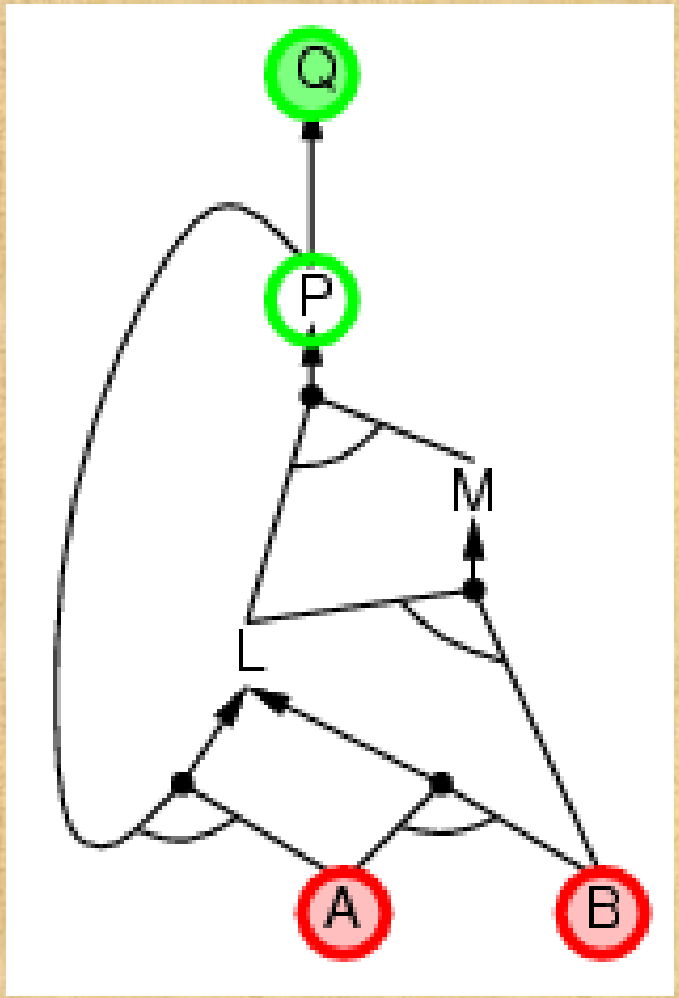
Avoid repeated work: check if new sub-goal

1. has already been proved true, or
2. has already failed

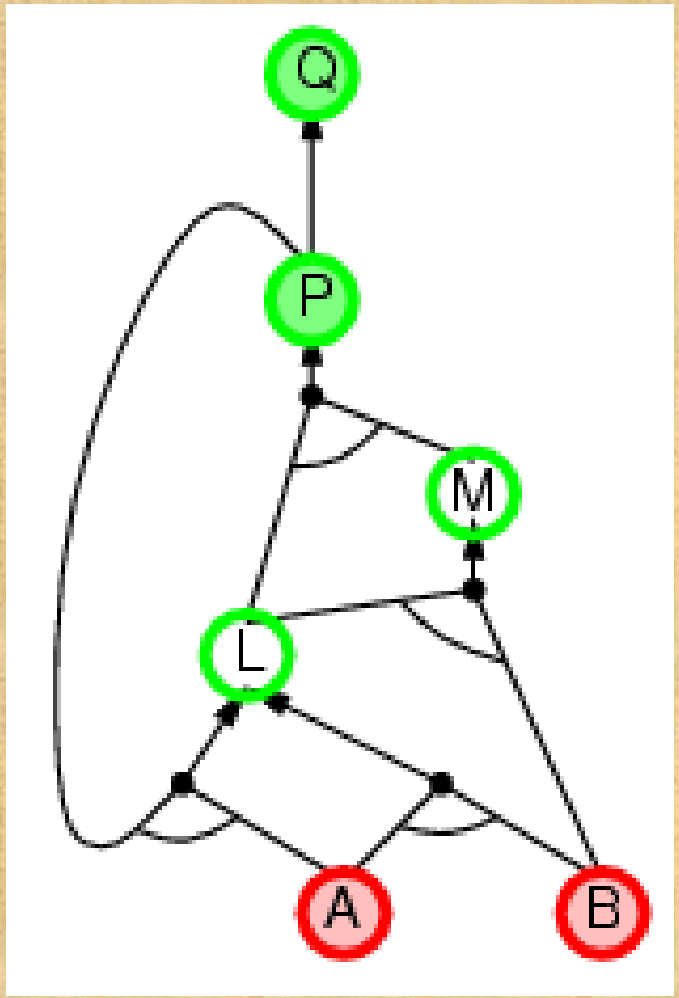
Backward chaining example



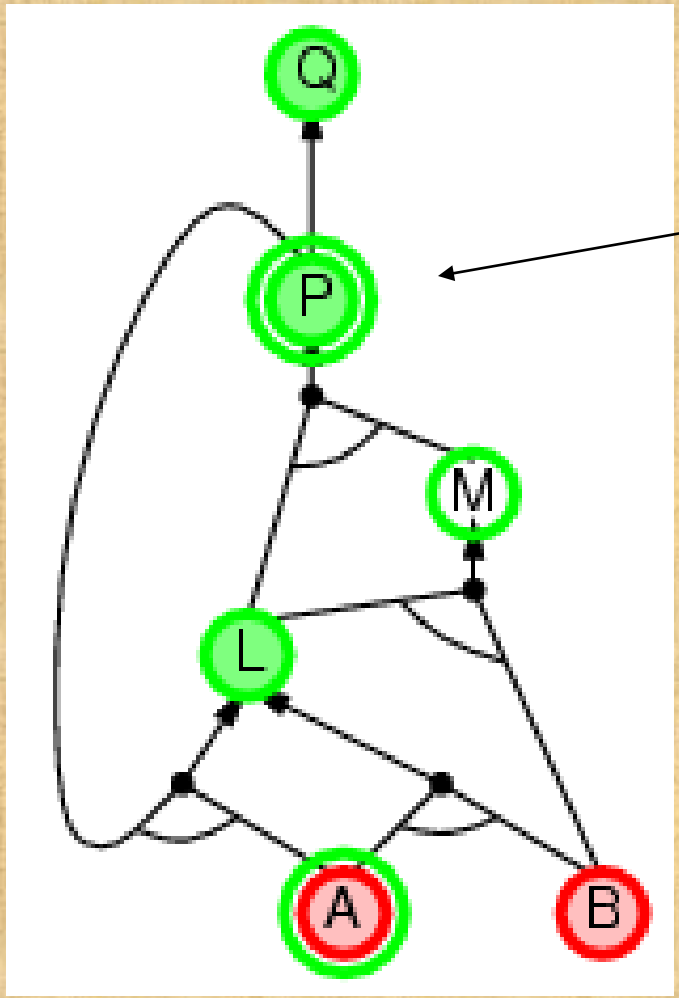
Backward chaining example



Backward chaining example

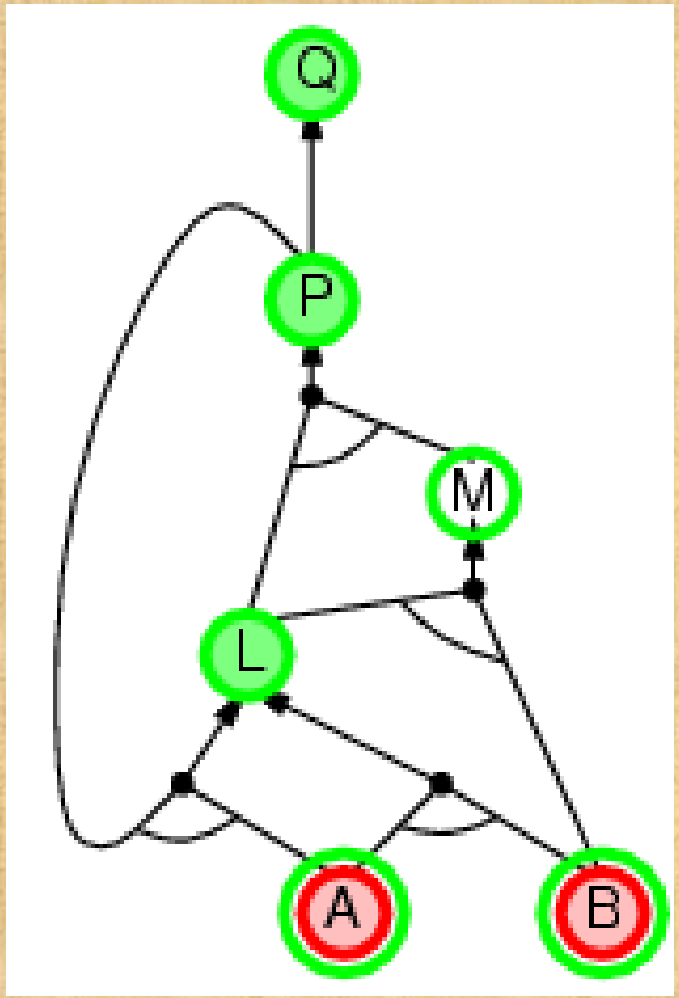


Backward chaining example

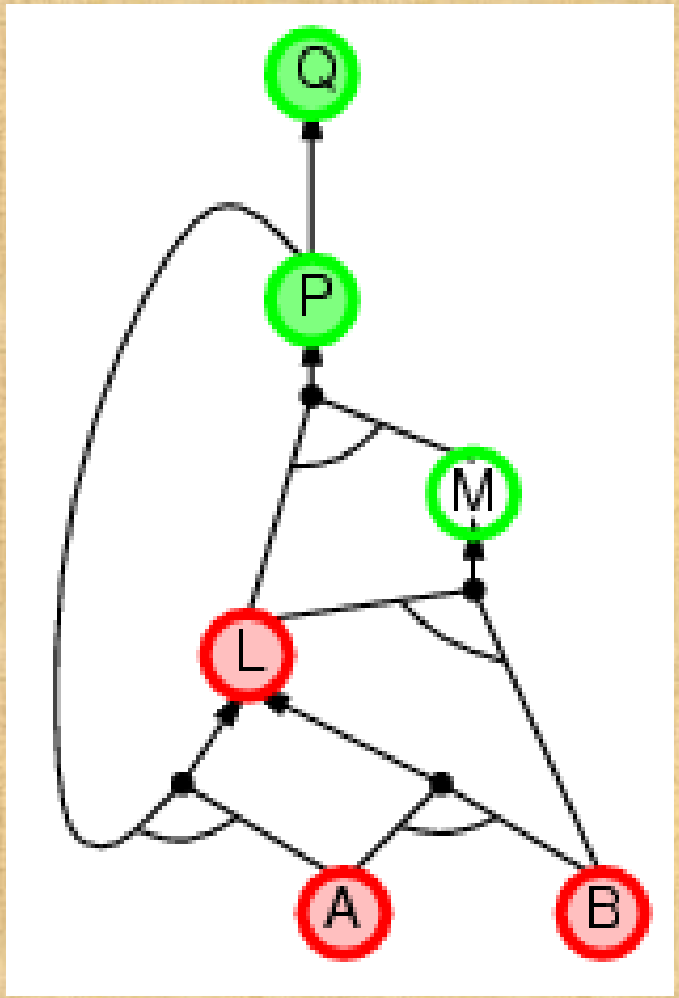


we need P to prove
L and L to prove P.

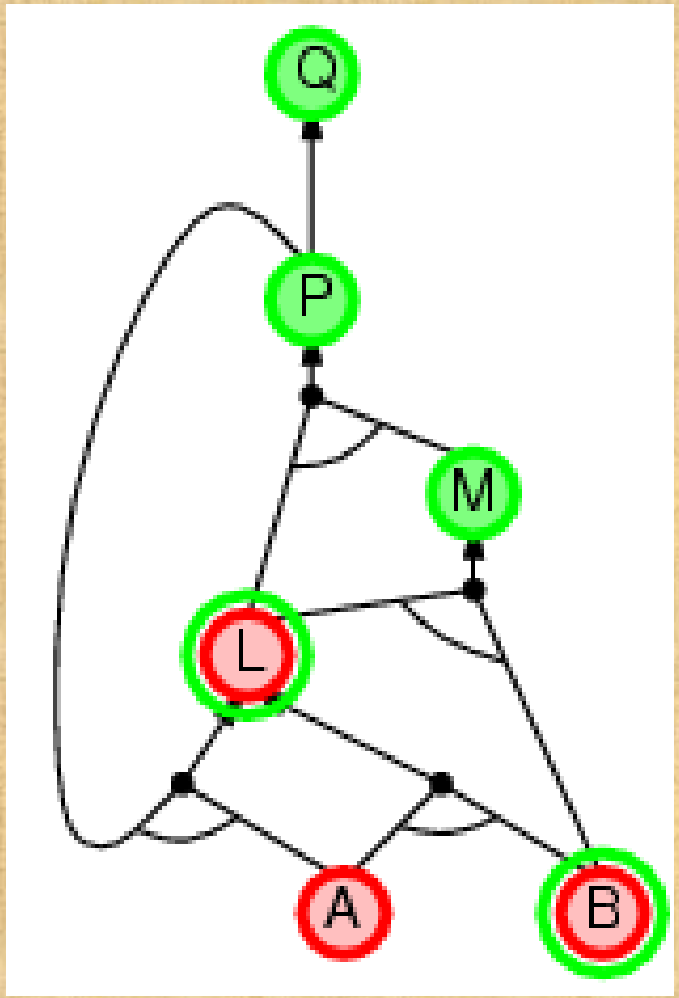
Backward chaining example



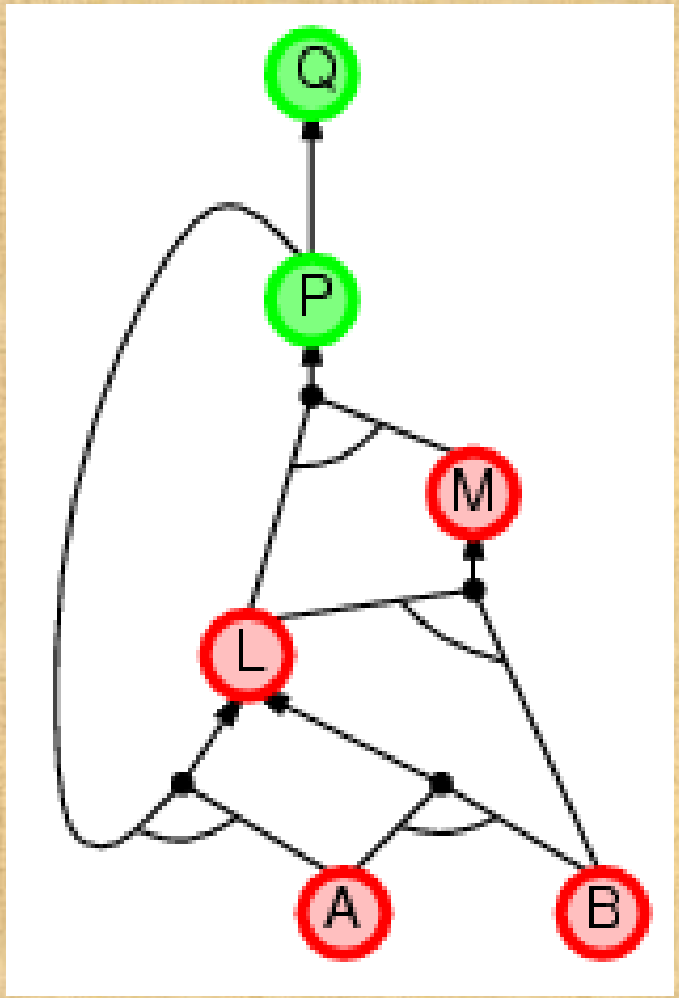
Backward chaining example



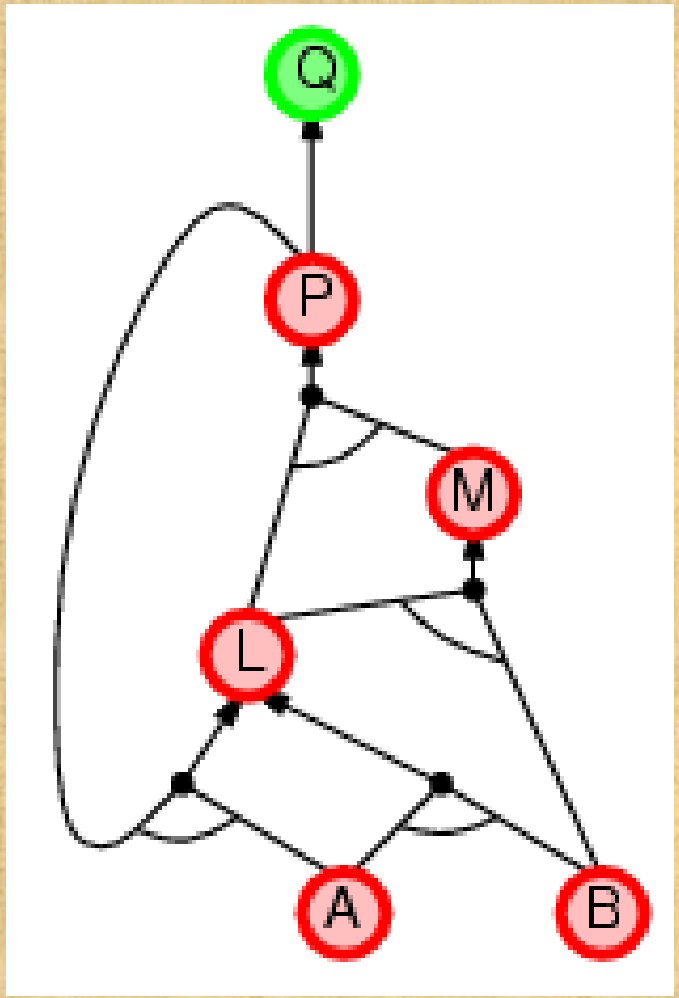
Backward chaining example



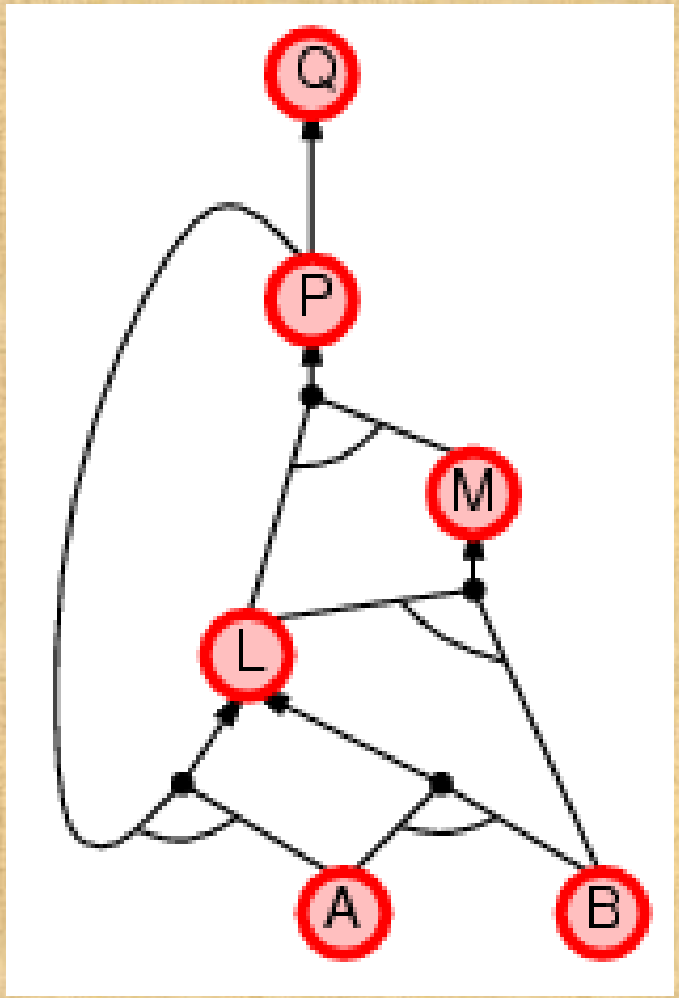
Backward chaining example



Backward chaining example



Backward chaining example



Demo: FC vs. BC

- FORWARD CHAINING
- BACKWARD CHAINING

Forward vs. backward chaining

- Forward chaining is **data-driven**, automatic, unconscious processing,
  - e.g., object recognition, routine decisions
- May do lots of work that is irrelevant to the goal
- Backward chaining is **goal-driven**, appropriate for problem-solving,
  - e.g., Where are my keys? How do I get into Windows Device Manager?
- Complexity of Backward chaining can be <u>much less</u> than linear in size of Knowledge Base

FC & BC in First-Order Logic

An Example: The law says that it is a crime for an American to sell weapons to hostile nations. The country Nono, an enemy of America, has some missiles, and all of its missiles were sold to it by Colonel West, who is American.

- ... it is a crime for an American to sell weapons to hostile nations:

$$American(x) \wedge Weapon(y) \wedge Sells(x,y,z) \wedge Hostile(z) \Rightarrow Criminal(x)$$

- Nono … has some missiles, i.e., $\exists x$ Owns(Nono,x) $\wedge$ Missile(x):

$$Owns(Nono,M1) \text{ and } Missile(M1)$$

- … all of its missiles were sold to it by Colonel West

$$Missile(x) \wedge Owns(Nono,x) \Rightarrow Sells(West,x,Nono)$$

- Missiles are weapons:

$$Missile(x) \Rightarrow Weapon(x)$$

- An enemy of America counts as "hostile“:

$$Enemy(x,America) \Rightarrow Hostile(x)$$

- West, who is American …

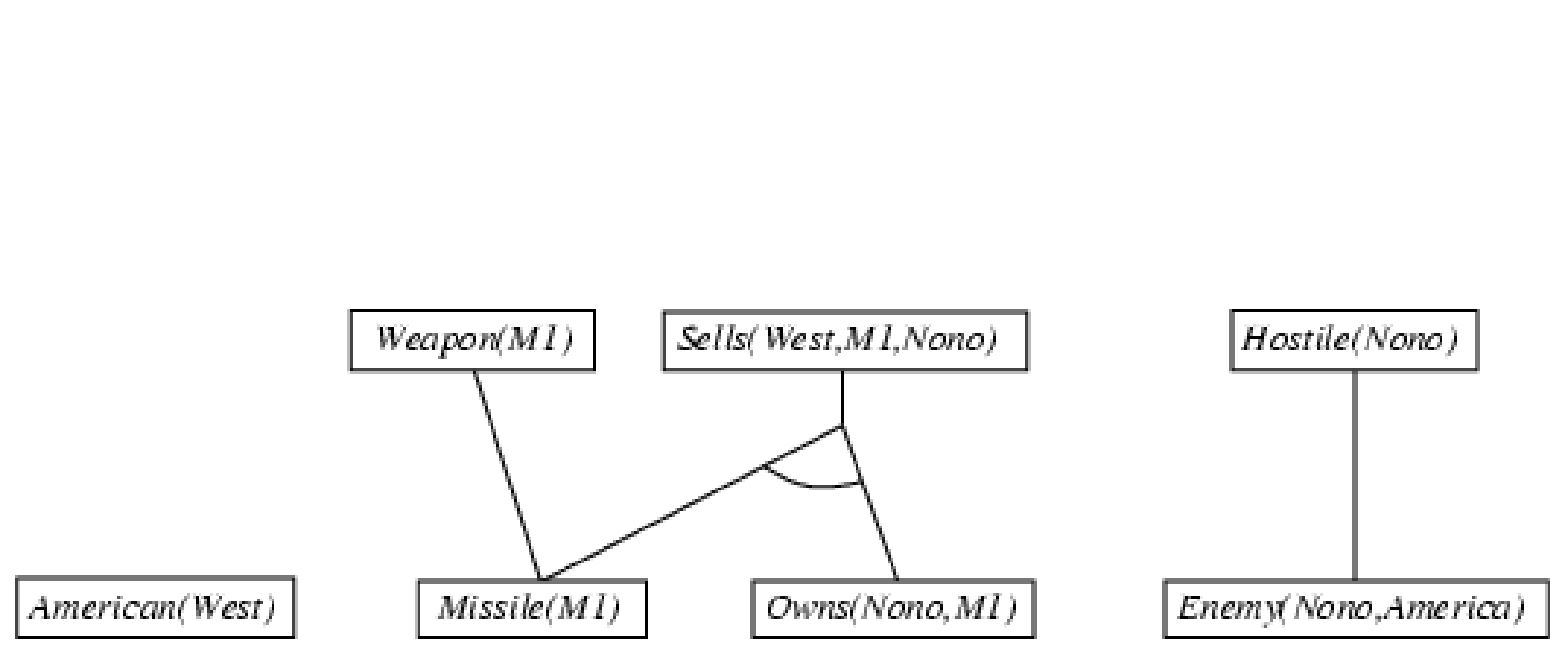
$$American(West)$$

- The country Nono, an enemy of America …

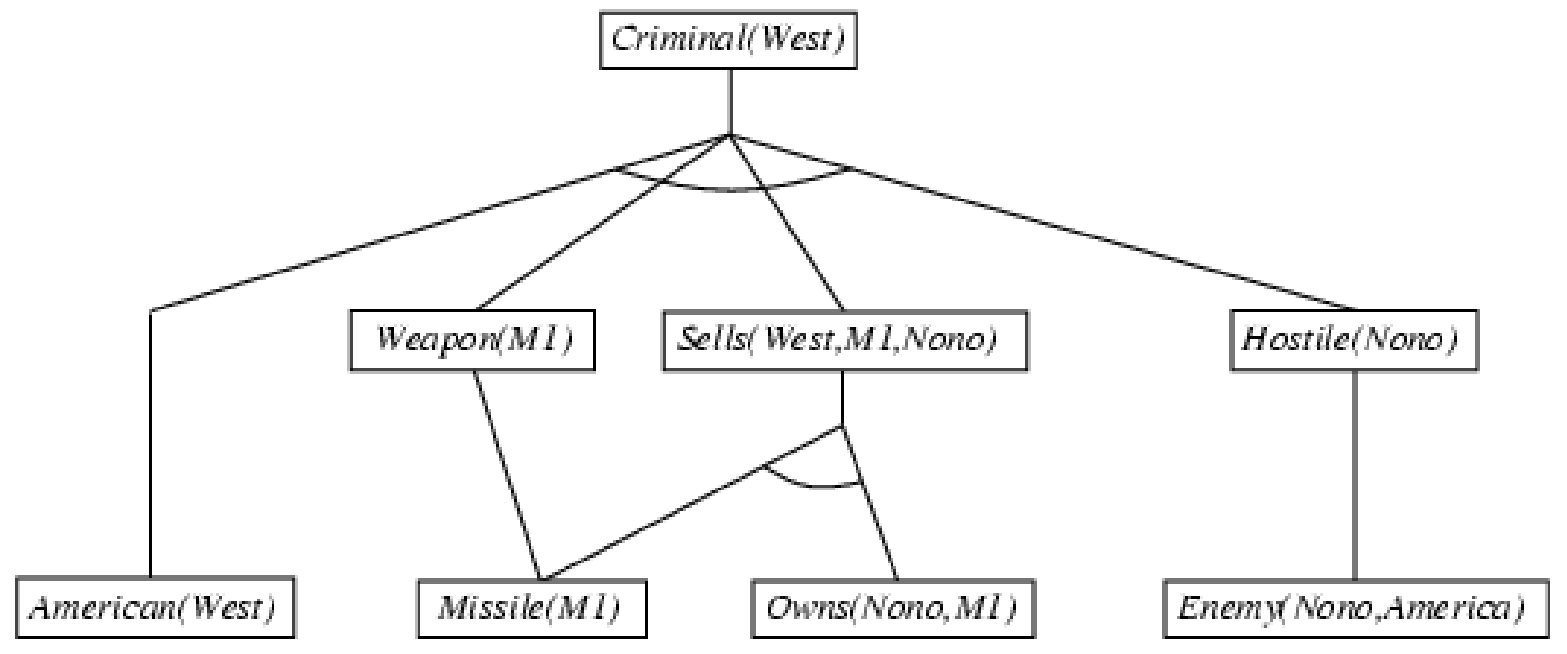
$$Enemy(Nono,America)$$

Forward chaining proof

Forward chaining proof



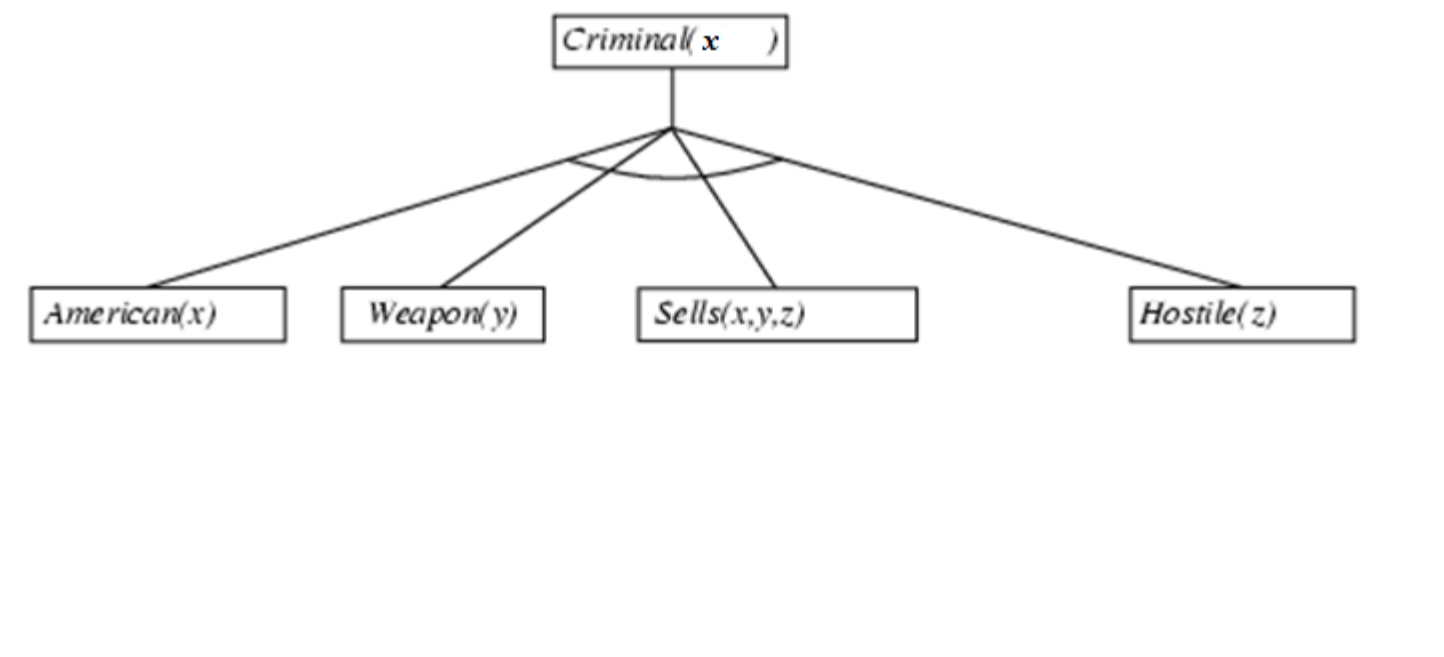
Forward chaining proof



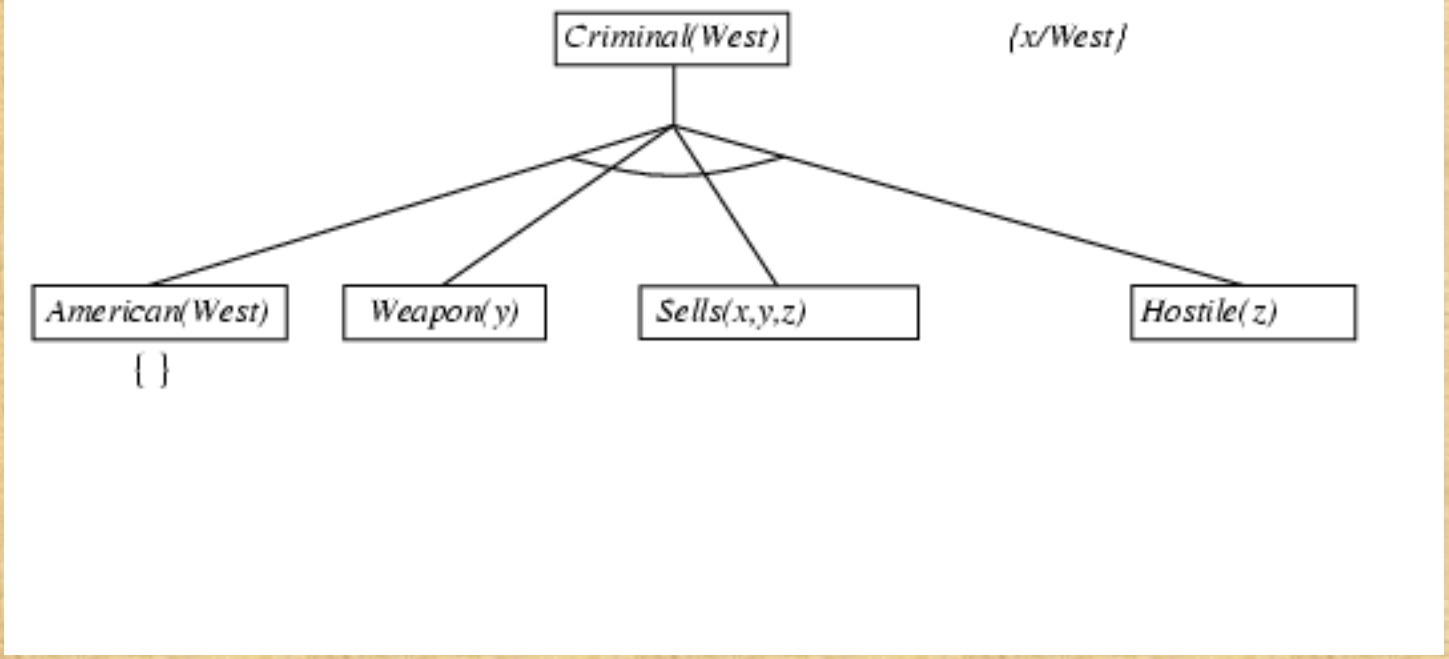
Backward chaining example

Criminal(West)

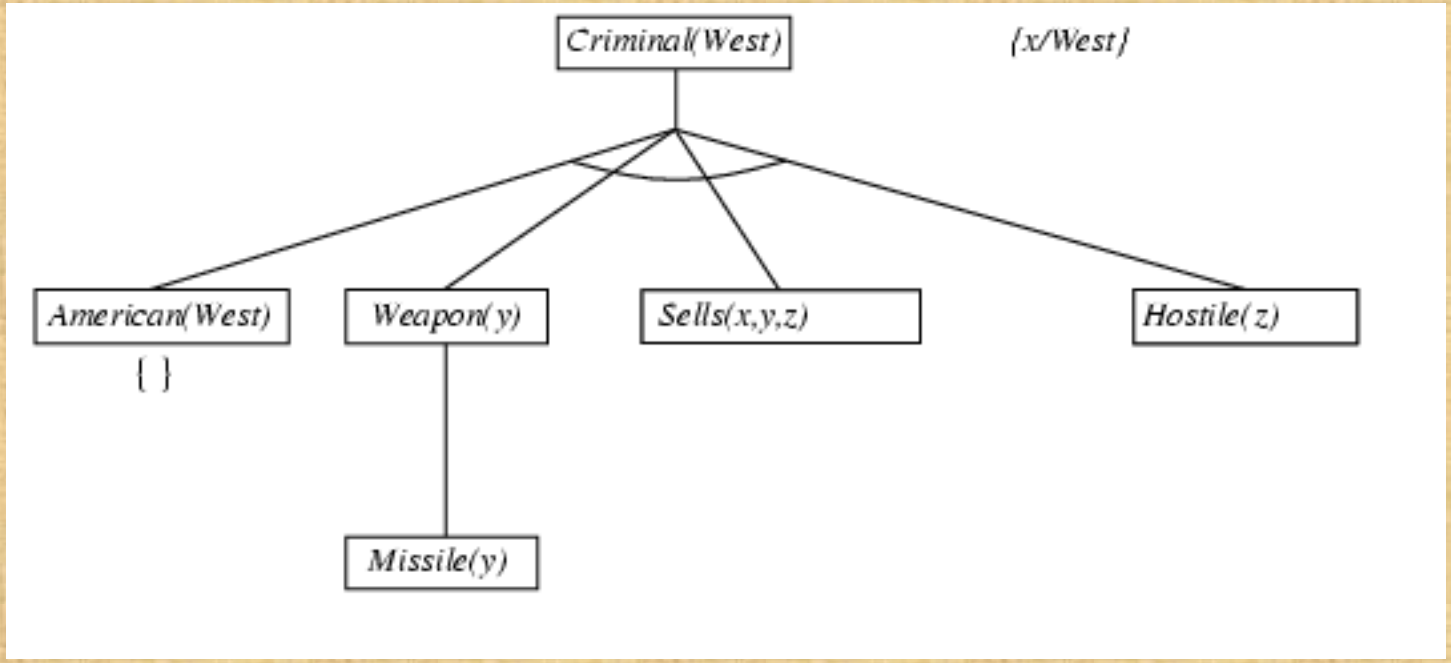
Backward chaining example



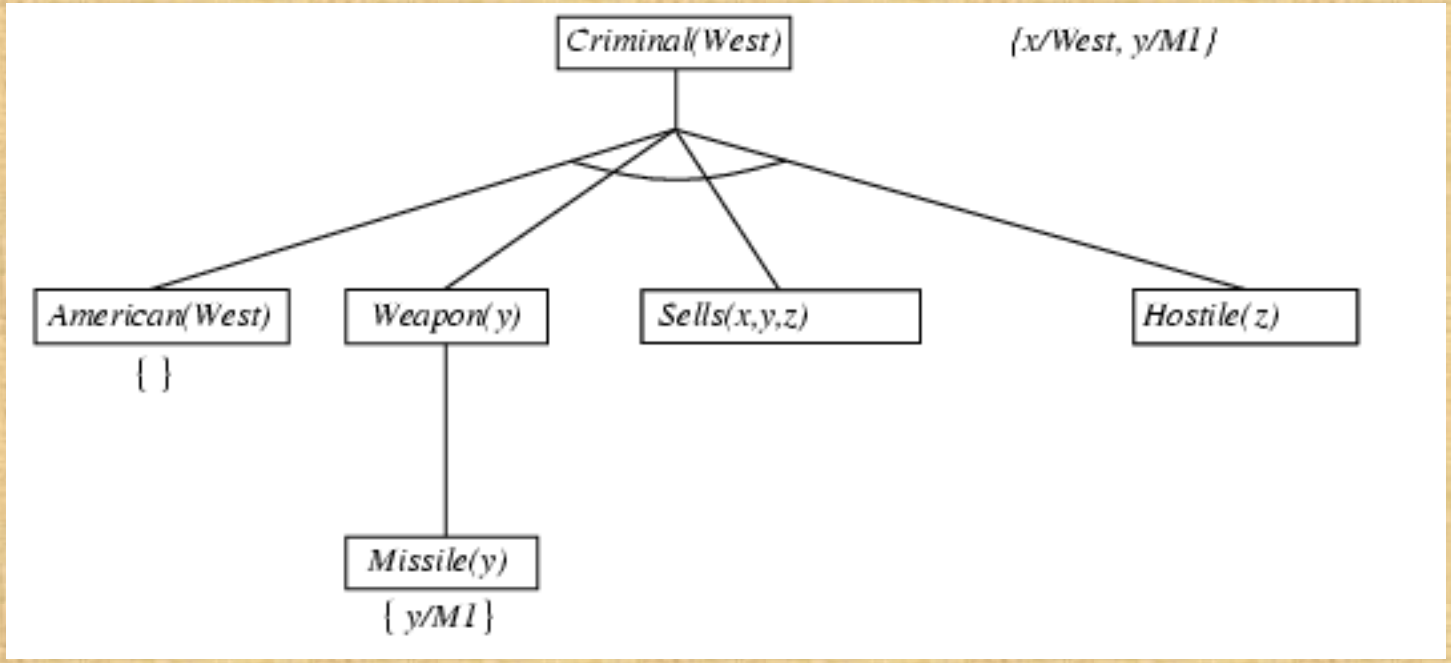
Backward chaining example



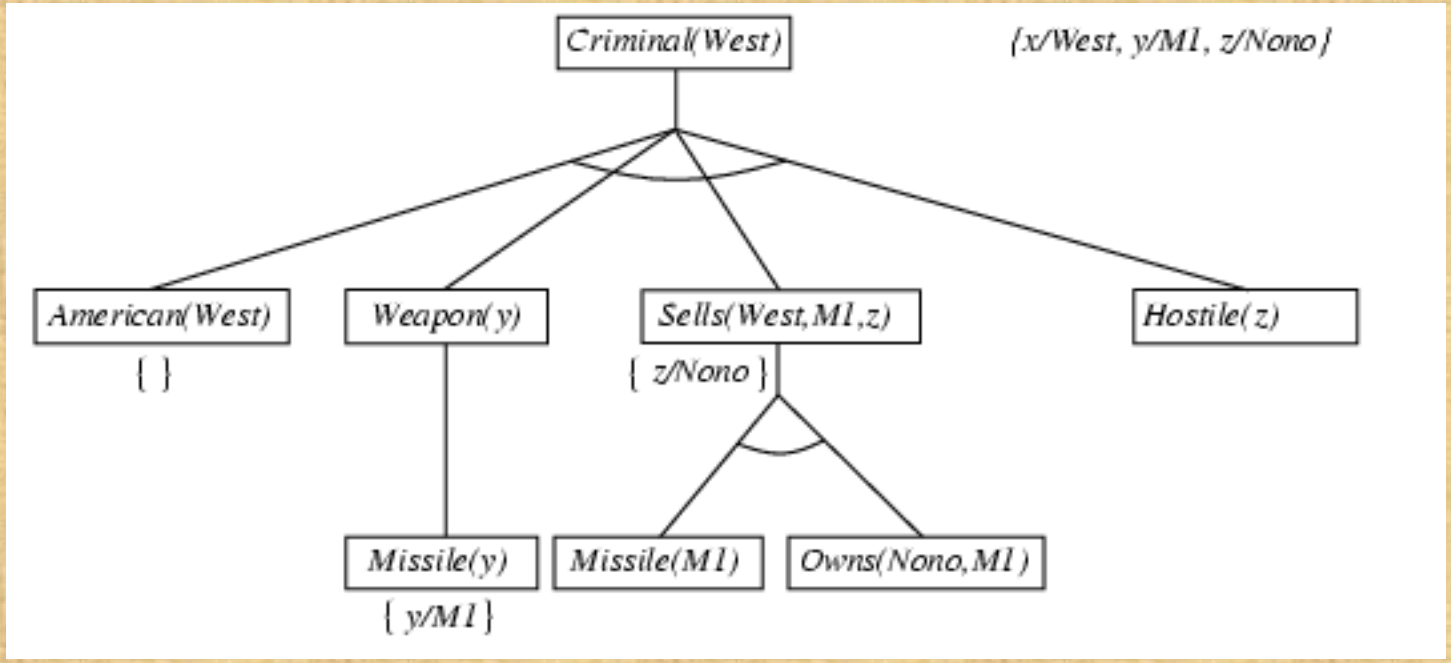
Backward chaining example



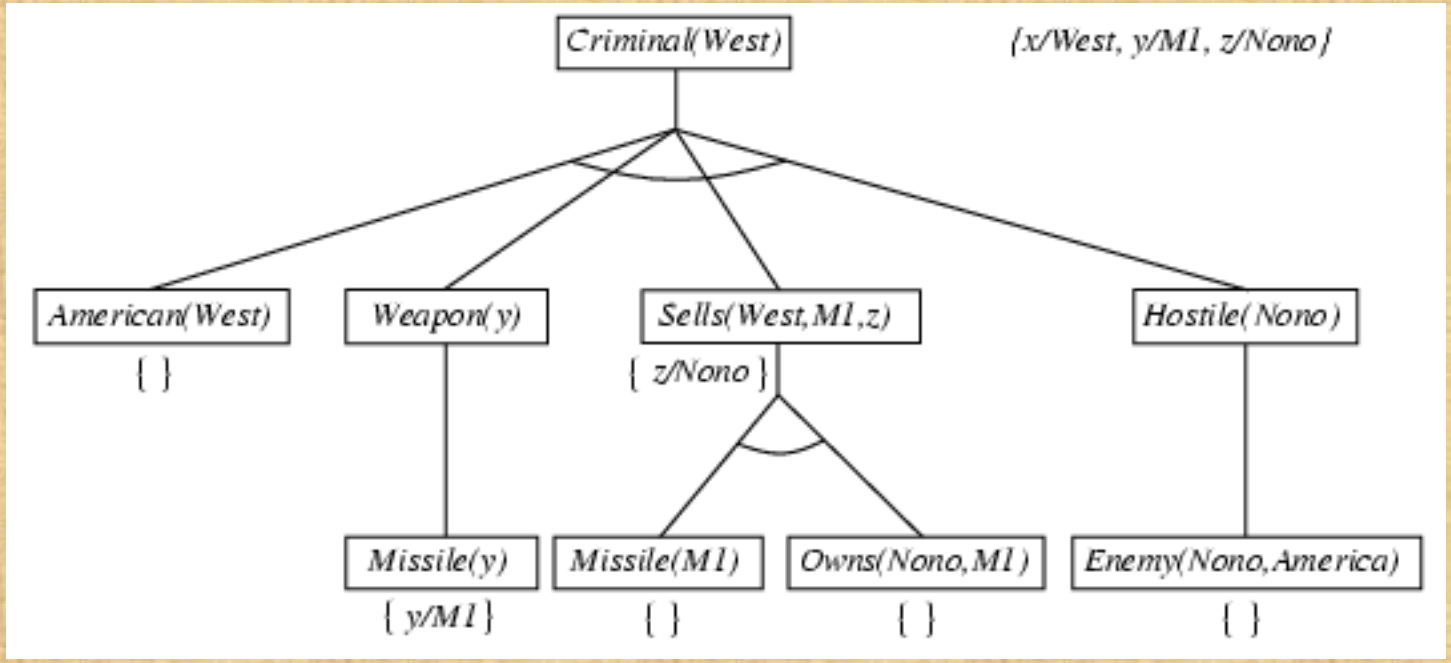
Backward chaining example



Backward chaining example



Backward chaining example



Knowledge engineering

- **Knowledge engineering** or **knowledge acquisition** is the process of codifying a human expert's expertise and representing that expertise in the knowledge base.

Knowledge engineering (con’t)

- The key to developing expert systems is to begin with the objectives of the consultation and then work backward to define user inputs and rules.
- Steps:
  1. Choosing an expert
  2. Capturing the expert's expertise
  3. Encoding this expertise as rules in a knowledge base.

Choosing an expert who…

- has experience in actually solving the problem.
- nobody wants you to have.
- a good communicator.

Explanations

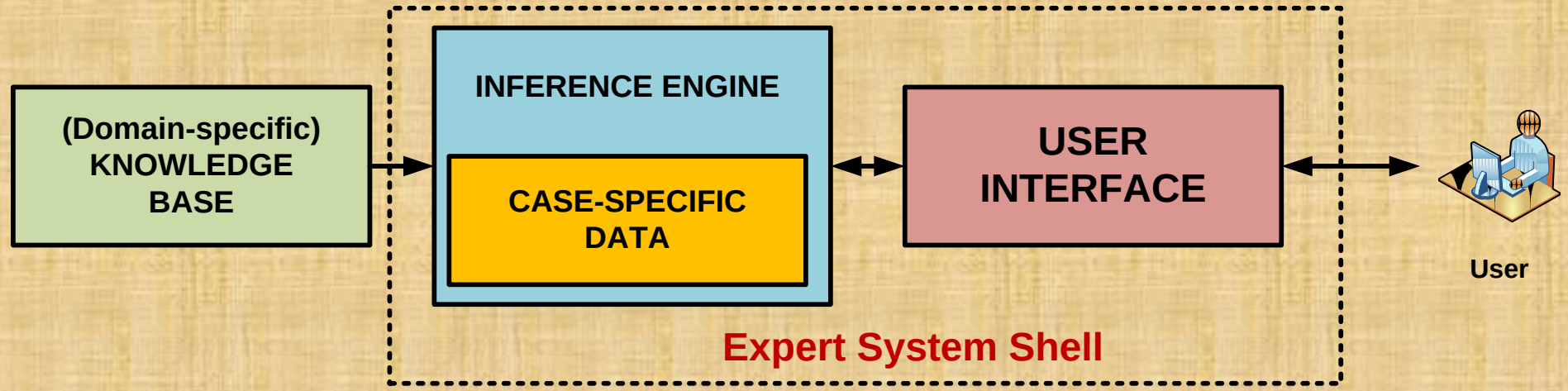
Explanation facilities are imperative for acceptance

- TEIRESIAS (1977) front-end for MYCIN, supported knowledge acquisition and explanation
- Could answer

WHY	Is that knowledge relevant
HOW	did it come to that conclusion
WHAT	Is it currently trying to show

Can add new rules and adjust existing rules

Expert system shell

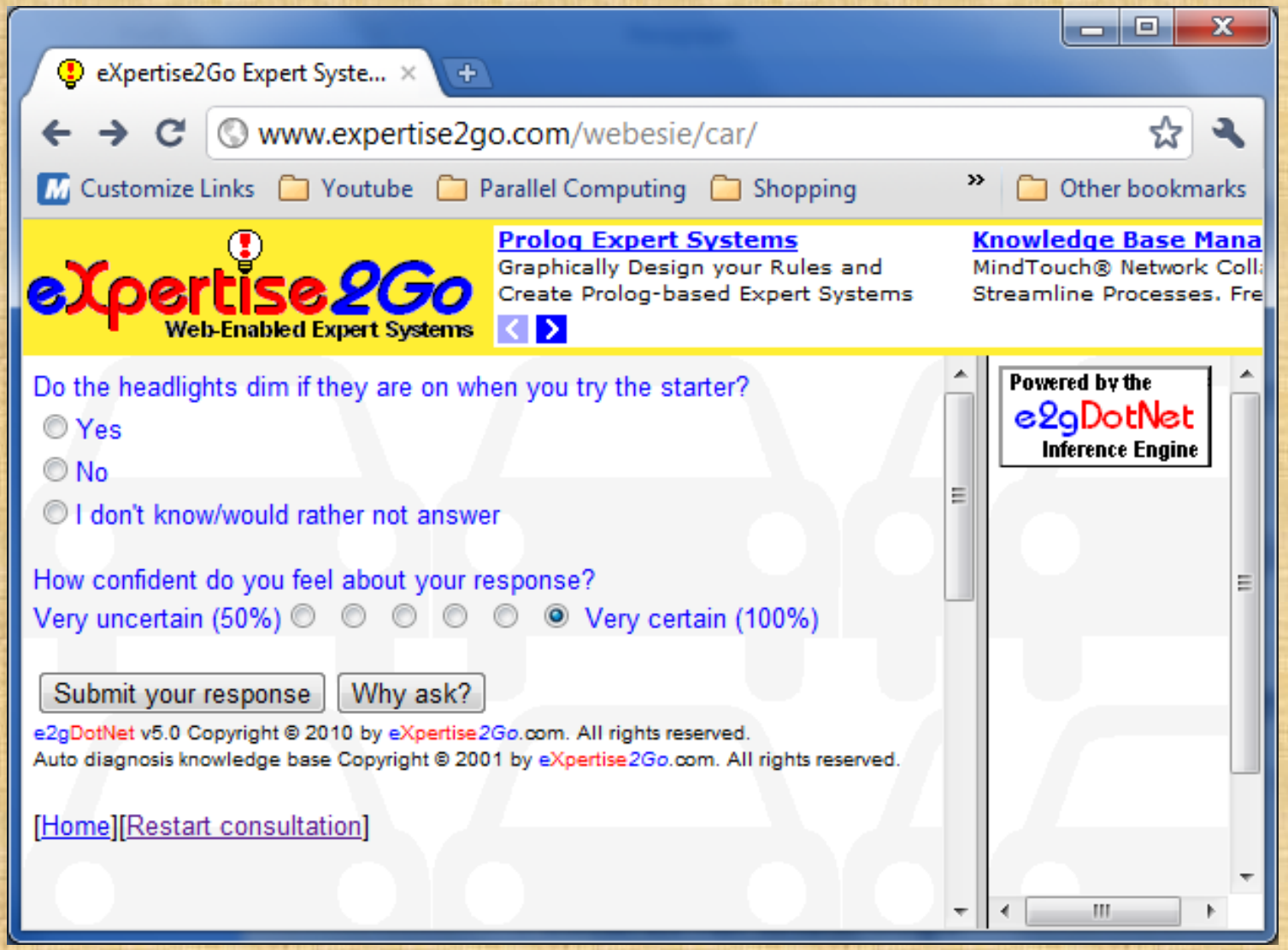


Expert System Shells are have <u>empty</u> knowledge base

Expert system shell

- eXpertise2Go provides a free expert system shell
  - The shell provides the inference engine, acquisition/explanation interface
  - User must insert the relevant knowledge base
  - **e2gRuleWriter** decision table software to create, simplify and validate **e2gRuleEngine** Knowledge Bases

eXpertise2Go



DEMO

- eXpertise2Go
- e2gRuleEngine
  - Budget Wine Selector
  - Auto Diagnosis Advisor
- e2gRuleWriter
- InterModeller

Advantages

- Consistent answers for repetitive decisions, processes and tasks
- Holds and maintains significant levels of information
- Encourages organizations to clarify the logic of their decision-making
- Never "forgets" to ask a question, as a human might

Disadvantages

- Lacks common sense
- The GIGO phenomenon: Errors may occur in the knowledge base
- Cannot adapt to changing environments
- Cannot make creative responses as human expert

Q & A

(If we have time, "How to handle uncertainty", otherwise)

How to handle uncertainty

- Certainty measurements are usually called **certainty factors** or **confidence factors** and denoted **CF**.
- Expert system implementations handle this problem in different ways:
  - often by letting users represent their degree of confidence in a response with a numerical scale.

How to handle uncertainty (an example)

Example	Method	Definition	Result
IF a AND b CFa=60% CFb=80%	Minimum	Use smaller of CF1, CF2	Premise is true, CF=60%
IF a OR b CFa=60% CFb=80%	Maximum	Use larger of CF1, CF2	Premise is true, CF=80%
CFa=70% from 1st source CFa=60% from 2nd source	Probability Sum	CF1 + CF2/100 x (100-CF1)	Final CFa=88% 70+(60/100)x(100-70)
IF a AND b Then c=5 with 80% confidence (premise CF=90%)	Multiplication	(CF1 x CF2)	c is 5 with 72% confidence

Thank You!